

Introduction To Linked List

Whenever we go to the departmental store we carry a list along with us. The list usually contains a list of items arranged in order indicating the items of our requirement. Many problems in computing also involve lists of items. A list is a data type, which contains elements of the same type arranged in a set of continuous memory locations. With lists it is possible for us to perform certain operations such as searching the list, sorting it, printing it, and so on. The structure we have used as the concrete data representation of a list is the array, which is a sequential structure.

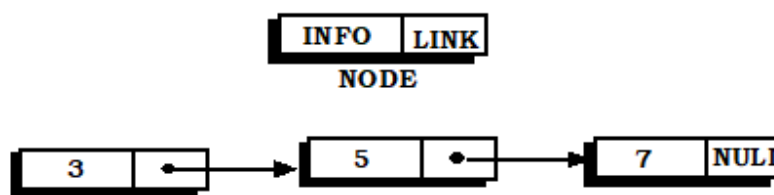
Operations such as searching can be very efficiently carried out using an array representation. However, inserting and deleting items cannot be easily performed with an array representation. To insert a new item into a location in a list, we must shift the array elements down to make room for the new item. Similarly, deleting an item from the list requires that we shift up all the array elements following the one to be deleted.

Another drawback with arrays is that they are not very adaptable since the size of the array must be fixed in advance. The size of the array cannot be increased or decreased during the execution of the program. These problems can be overcome by considering another data structure called - the **linked list**.

LINKED LIST

A linked list represents a linear collection of items, called as nodes. Nodes of a linked list can be scattered about in the memory, they need not necessarily represent a set of consecutive memory locations. Each node in a linked list has two parts:

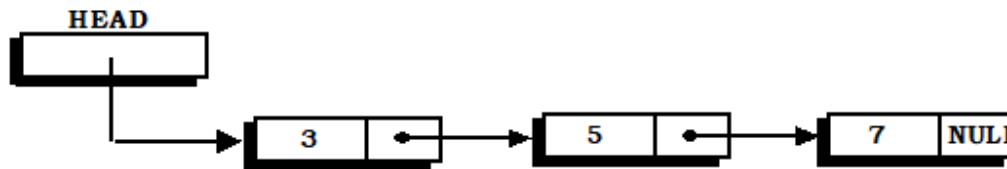
1. **Information part**, which contains the data for the node.
2. **Link part**, which gives the memory address of the next node in the list.



The Figure shows an abstract representation of a linked list. Information part may consist of one or more fields. A linked list thus consists of a series of structures, which are not necessarily contiguous. Each structure contains an information field and a link, which is a pointer to the next structure, which is its successor. An arrow is used in the link member of each node to indicate the location of the next node. The **NULL** in the link member of the last node signifies the end of the list.

SINGLY LINKED LIST

A linked list in its simplest form is a collection of nodes that together form a linear ordering. The ordering is determined as in the children's game "**Follow the leader**," in that each node usually consists of a structure that includes information fields and a link pointer. A singly linked list looks like that shown in the Figure.



The Figure shows a list of numbers, which is represented, in the form of a linked list. **HEAD** is a pointer, which gives the address of the first node in the linked list. Link field of the last node contains the **NULL**, which indicates it does not contain any address.

It might seem like circular reasoning to have a node reference another node, but such a scheme easily works. The next reference inside a node can be viewed as a link or pointer to another node. Likewise, moving from one node to another by following a next reference is known as **link hopping** or **pointer hopping**. The first and last node of a linked list usually is called the **HEAD** and **TAIL** of the list, respectively. We identify the tail as the node having a **NULL** next reference, which indicates the termination of the list. A linked list defined in this way is known as a **singly linked list**.

We have seen that a linked list is a collection of nodes of the same type and hence and it can be defined as follows:

```
struct Node
{
    Data type    Information;
    struct Node *Link;
};
```

Definition of a NODE of a Linked list in C

Once the structure of a node is defined we may want to allocate memory space for the nodes of the linked list. Memory space is always allocated with the help of a declaration statement. If we want to create a linked list that contains three nodes called **Node1**, **Node2** and **Node3** the declaration may be as follows.

```
struct node *Node1,*Node2,*Node3;
```

DIFFERENT OPERATIONS POSSIBLE ON A LINKED LIST

1. Creating a linked list.
2. Traversing a linked list.
3. Inserting an item into a linked list.
4. Deleting an item from the linked list.

5. Searching an item in a linked list.
6. Merging two or more lists to form a single list.

CREATING A LINKED LIST

Using the above program we were able to create a linked list, which contains three nodes. There are some situations where we do not know how many elements are to be inserted in a linked list. In such a situation we will have to use an alternative technique in which the list components are dynamically created only as they are needed.

These dynamic nodes are created by using pointers and dynamic memory allocation function such as **malloc()**. We start accessing the list with a pointer variable that holds the address of the first node in the list. This pointer variable is named as **HEAD**. Every node after the first node can be accessed by using the link member in the previous node.

Such a list can expand or contract as the program executes. To insert a new item into the list, we allocate more space. To delete an item, we de-allocate the memory assigned to it. We don't have to know in advance how long the list will be. The only limitation is the amount of available memory space. Data structures built using this technique are called **dynamic data structures**.

To create a dynamic linked list, we begin by initializing the header **HEAD** to **NULL**. We then create the first node and then assign its address to the header **HEAD**. We then allocate a second node and store its address in the link part of the first node. We continue this process of allocating a new node and storing the pointer to it into the link part of the previous node the user decides to stop the process.

ALGORITHM:

```

Step 1 :      [Create a new node and store its address in the pointer NewNode]
              NewNode = Address of new node
Step 2 :      [Copy the information for the new node to the information part]
              INFO[NewNode] = element
Step 3 :      [Set the contents of the link part as NULL]
              LINK[NewNode] = NULL
Step 4 :      [Is the new node the first node ?]
              If answer is yes Then
                [Connect the new node to the header ]
                HEAD = NewNode
              Else
                (a)  [Take the CurrPtr to the last node of the linked list]
                    CurrPtr = Address of the last node
                (b)  [Connect the new node to the link of the last node]
                    LINK[CurrPtr] = NewNode
                (c)  Exit
              [ Endif ]
Step 5 :      Return

```

In the above algorithm, we used three pointers:

1. **HEAD** is used to hold the address of the first node of the linked list.
2. **NewNode** is used to hold the address of the new node.
3. **CurrPtr**, which is updated to always point to the last node in the linked list.

Program: Program to create a linked list.

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <conio.h>
struct node
{
    int info;
    struct node *link;
};
typedef struct node NODE;
NODE *HEAD;

void Create_link_list(int);
void Display(NODE *);

void main()
{
    int ele;
    char ch;

    clrscr();
    HEAD = NULL;
    printf("\nLinked list creation\n\n");
    do{
        printf("Enter information field of the node\n");
        scanf("%d",&ele);
        Create_link_list(ele);
        printf("Add another node\n");
        fflush();
        scanf("%c",&ch);
    }while(toupper(ch)!='Y');
    printf("\nThe linked list is \n");
    Display(HEAD);
    getch();
}

/* Function to add a node to the linked list */
void Create_link_list(int ele)
{
    NODE *NewNode,*CurrPtr;
```

```

        NewNode=(NODE *)malloc(sizeof(NODE));
        NewNode->info=ele;
        NewNode->link=NULL;

        if( HEAD==NULL)
            HEAD=NewNode;
        else
        {
            CurrPtr=HEAD;
            while(CurrPtr->link!=NULL)
                CurrPtr=CurrPtr->link;

            CurrPtr->link=NewNode;
        }
    }
    /*      Function to display linked list elements      */
    void Display(NODE *CurrPtr)
    {
        while(CurrPtr!=NULL)
        {
            printf("%d\t",CurrPtr->info);
            CurrPtr=CurrPtr->link;
        }
        printf("NULL\n");
    }
}

```

Let us go through some of the statements, describing in words what is happening and showing the linked list as it appears after the execution of the statement.

HEAD = NULL;

The statement is used to initialize the header to NULL, which indicates that the linked list has not yet been created.

NewNode = (NODE *)malloc(sizeof(NODE));

A dynamic variable of type NODE is created. This statement obtains a piece of memory to store a node and assigns its address to the pointer variable **NewNode**.

NewNode->info = ele;

NewNode->link=NULL;

Using the above statements, the contents of the new node are entered.

HEAD = NewNode;

Connects the first node to the header pointer

```

CurrPtr=HEAD;
while(CurrPtr->link!=NULL)
    CurrPtr=CurrPtr->link;

```

The looping statement takes the CurrPtr from the first node to the last node of the linked list.

```

CurrPtr->link=NewNode;

```

The pointer to the new node is copied into the link member of CurrPtr, which connects the newnode to the linked list.

TRAVERSING A LINKED LIST

Traversing is the process of visiting each node of the linked list exactly once to perform some operation. Our aim is to traverse the list starting from the first node to the end of the list. Let LIST be a linked list with nodes containing two fields INFO and LINK. HEAD is a pointer variable, which contains the address of the first node in the linked list. CurrPtr is a pointer variable, which points to the node currently being processed. The fields of the node is referenced by INFO[CurrPtr] indicates information field of the node which has the address stored in CurrPtr variable. LINK[CurrPtr] is the link field of the node which has the address stored in CurrPtr variable. The statement

```

CurrPtr = LINK[CurrPtr]

```

moves the pointer to the next node in the list as illustrated in the figure below.

ALGORITHM:

This algorithm traverses a linked list by applying a process to each element of the list. The variable **CurrPtr** points to the node currently being processed.

```

Step 1 : [ Initialize pointer variable CurrPtr]
          CurrPtr = HEAD
Step 2 : [ Perform the traversing Operation ]
          While CurrPtr != NULL Do
Step 3 : Apply PROCESS to INFO[CurrPtr]
Step 4 : [ Move pointer to next node ]
          CurrPtr = LINK[CurrPtr]
          [End of While Operation]
Step 5 : Exit

```

Function to perform traversing

```

void Display(NODE *CurrPtr)
{
    while(CurrPtr != NULL)
    {
        printf("%d\t", CurrPtr->info);
        CurrPtr=CurrPtr->link;
    }
}

```

```

    }
    printf(" NULL\n");
}

```

INSERTING AN ITEM INTO A LINKED LIST

There are many applications where it is desired to maintain an order linear list. The ordering may be in increasing order or decreasing order of information field. Such an ordering often results more efficient processes in many cases. Before inserting a new node, first we have to take the free node from the available list. If no free nodes are available then we cannot insert the node. There are different types of insertions.

1. Insert a Node at the beginning of the linked list.
2. Insert a Node at the end of the linked list.
3. Insert a Node at a given position in the list

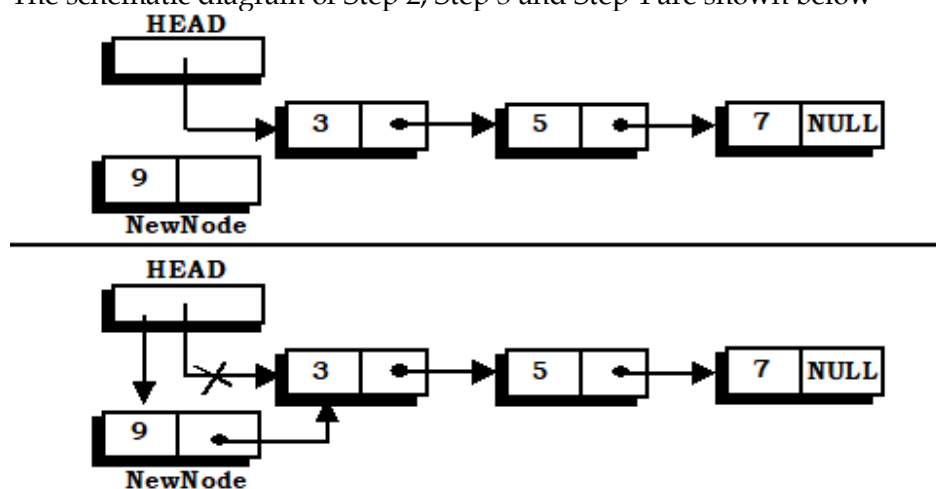
INSERT A NODE AT THE BEGINNING OF THE LINKED LIST

Let the **HEAD** pointer hold the address of the first node of the linked list. The following algorithm will create a new node and insert this node at the beginning of the linked list.

ALGORITHM:

- Step 1 : [Create a new node and store its address in the pointer NewNode]
 NewNode = Address of new node
- Step 2 : [Copy the information for the new node to the information part]
 INFO[NewNode] = element
- Step 3 : [Set the contents of the link part to contain the address of the first node]
 LINK[NewNode] = HEAD
- Step 4 : [Make the new node as the first node of the linked list]
 HEAD = NewNode
- Step 5 : Exit

The schematic diagram of Step 2, Step 3 and Step 4 are shown below



Function to insert a node at the beginning of the linked list

```
void Insert_Beginning( int ITEM )
{
    NODE *NewNode;

    NewNode= (NODE *) malloc(sizeof(NODE));

    NewNode->info=ITEM;
    NewNode->link=HEAD;
    HEAD = NewNode;

}
```

INSERT A NODE AT THE END OF THE LINKED LIST

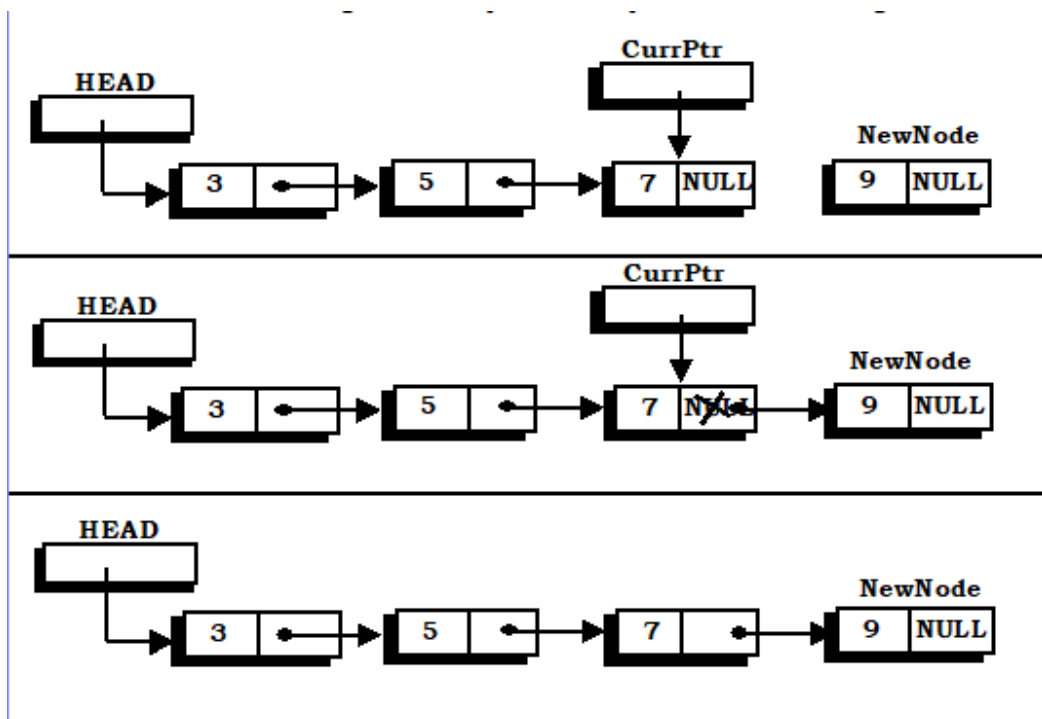
In some of the applications we have to append an item at the end of the list. To append an item to a linked list, we loop through the list until find the element whose link member points to NULL. The following algorithm is used to insert a node into a linked list as a last node.

ALGORITHM:

Let **LIST** be a list stored in memory, **HEAD** gives the address of the first node. This algorithm inserts **ITEM** into the list as the last node. **CurrPTR** is a pointer, which is pointing to the current node of the list.

- Step 1 : [Create a new node and store its address in the pointer NewNode]
NewNode = Address of new node
- Step 2 : [Copy the information for the new node to the information part]
INFO[NewNode] = element
- Step 3 : [Set the contents of the link part to NULL]
LINK[NewNode] = NULL
- Step 4 : [Initialize HEAD value to CURRPTR]
CurrPtr = HEAD
- Step 5 : [Find the last node of the list]
While LINK[CurrPtr] != NULL Do
- Step 6 : [Move CURRPTR to next node]
CurrPtr = LINK[CurrPtr]
[WhileEnd]
- Step 7 : [Connect the new node to the link of the last node]
LINK[CurrPtr] = NewNode
- Step 8: Exit

The schematic diagram of Step 6 and Step 7 are shown in Figure



Function to insert a node at the end of the linked list

```
void Insert_End (int ITEM)
{
    NODE *NewNode, *CurrPtr;

    NewNode= (NODE *) malloc(sizeof(NODE));
    NewNode->info=ITEM;
    NewNode->link=NULL;
    CurrPtr=HEAD;
    while(CurrPtr->link!=NULL)
        CurrPtr=CurrPtr->link;

    CurrPtr->link=NewNode;
}
```

INSERT A NODE AT A GIVEN POSITION

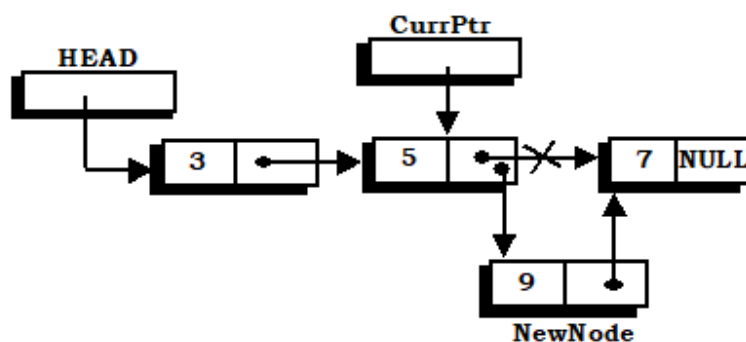
Suppose we are given the ITEM, which has to be placed at a given position in the linked list. The following is an algorithm, which inserts ITEM into LIST so that ITEM is placed at the required position.

ALGORITHM:

Let **LIST** be a list stored in memory, **HEAD** gives the address of the first node. This algorithm inserts **ITEM** into the list at the position **POS**. **CurrPTR** is a pointer, which is pointing to the current node of the list.

- Step 1 : [Create a new node and store its address in the pointer NewNode]
 NewNode = Address of new node
- Step 2 : [Copy the information for the new node to the information part]
 INFO[NewNode] = element
- Step 3 : [Initialize HEAD value to CURRPTR]
 CurrPtr = HEAD
- Step 4 : [Move to the required position]
 For I = 1 to POS - 1 and CurrPtr != NULL Do
- Step 5 : [Move CURRPTR to next node]
 CurrPtr = LINK[CurrPtr]
- [ForEnd]
- Step 6 : [Check whether the position is found or not]
 If (CurrPtr = NULL)
 Print " Position Out Of Range"
 Exit
- Step 7 : Else
 [Insert the new node at the required position]
 LINK[NewNode] = LINK[CurrPtr]
 LINK[CurrPtr] = NewNode
 [EndIf]
- Step 8: Exit

The schematic diagram of Step 4 and Step 7 are shown in Figure



Function to insert a node at a given position

```

void Insert_Position( int ITEM, int POS)
{
    NODE *NewNode,*CurrPtr;
    int i;

    NewNode= (NODE *) malloc(sizeof(NODE));
    NewNode->info=ITEM;
  
```

```

    CurrPtr = HEAD;
    for( i = 1; i< POS - 1 && CurrPtr != NULL; i++)
        CurrPtr = CurrPtr->link;

    if ( CurrPtr ==NULL)
    {
        printf("Position Out Of Range\n");
        exit(0);
    }
    NewNode->link = CurrPtr->link;
    CurrPtr->link = NewNode;
}

```

Program: To create a linked list and insert an element into it

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <conio.h>
struct node
{
    int info;
    struct node *link;
};
typedef struct node NODE;
NODE *HEAD;

void Create_link_list(int);
void Display(NODE *);
void Insert_Beginning(int);
void Insert_End(int);
void Insert_Position(int,int);

void main()
{
    int ele,pos;
    char ch;

    clrscr();
    HEAD = NULL;
    printf("\nLinked list creation\n\n");
    do{
        printf("Enter information field of the node\n");
        scanf("%d",&ele);
        Create_link_list(ele);
        printf("Add another node\n");
        fflush();
        scanf("%c",&ch);
    }
}

```

```

}while(toupper(ch)=='Y');
printf("\nThe linked list is \n");
Display(HEAD);

printf("\nInsert Operation\n\n");
do
{
    printf("\nChoose Position To Insert: \n");
    printf("\n1. AT THE BEGINING");
    printf("\n2. AT THE END");
    printf("\n3. AT A GIVEN POSITION");
    printf("\n4. EXIT");
    printf("\n\nEnter your choice : ");
    scanf("%d",&ch);

    switch(ch)
    {
        case 1: {
            printf("\nEnter element to add : ");
            scanf("%d",&ele);
            Insert_Beginning(ele);
            printf("\nThe linked list is \n");
            Display(HEAD);
            break;
        }
        case 2: {
            printf("\nEnter element to add : ");
            scanf("%d",&ele);
            Insert_End(ele);
            printf("\nThe linked list is \n");
            Display(HEAD);
            break;
        }
        case 3: {
            printf("\nEnter element to add : ");
            scanf("%d",&ele);
            printf("\nEnter position to add: ");
            scanf("%d",&pos);
            Insert_Position(ele,pos);
            printf("\nThe linked list is \n");
            Display(HEAD);
            break;
        }
    }
}while(ch != 4);
getch();
}

```

/* Function to add a node to the linked list */

```
void Create_link_list(int ele)
{
    NODE *NewNode,*CurrPtr;

    NewNode=(NODE *)malloc(sizeof(NODE));
    NewNode->info=ele;
    NewNode->link=NULL;

    if( HEAD==NULL)
        HEAD=NewNode;
    else
    {
        CurrPtr=HEAD;
        while(CurrPtr->link!=NULL)
            CurrPtr=CurrPtr->link;
        CurrPtr->link=NewNode;
    }
}
```

/* Function to display linked list elements */

```
void Display(NODE *CurrPtr)
{
    while(CurrPtr!=NULL)
    {
        printf("%d\t",CurrPtr->info);
        CurrPtr=CurrPtr->link;
    }
    printf("NULL\n");
}
```

/* Function to insert a node at the beginning of the linked list */

```
void Insert_Beginning( int ITEM )
{
    NODE *NewNode;

    NewNode= (NODE *) malloc(sizeof(NODE));

    NewNode->info=ITEM;
    NewNode->link=HEAD;
    HEAD = NewNode;
}
```

/* Function to insert a node at the end of the linked list */

```
void Insert_End (int ITEM)
{

```

```

    NODE *NewNode, *CurrPtr;

    NewNode= (NODE *) malloc(sizeof(NODE));
    NewNode->info=ITEM;
    NewNode->link=NULL;
    CurrPtr=HEAD;
    while(CurrPtr->link!=NULL)
        CurrPtr=CurrPtr->link;

    CurrPtr->link=NewNode;
}

/*    Function to insert a node at a given position in a linked list    */
void Insert_Position( int ITEM, int POS)
{
    NODE *NewNode,*CurrPtr;
    int i;

    NewNode= (NODE *) malloc(sizeof(NODE));
    NewNode->info=ITEM;

    CurrPtr = HEAD;
    for( i = 1; i< POS-1 && CurrPtr != NULL; i++)
        CurrPtr = CurrPtr->link;
    if ( CurrPtr ==NULL)
    {
        printf("Position Out Of Range\n");
        exit(0);
    }

    NewNode->link = CurrPtr->link;
    CurrPtr->link = NewNode;
}

```

DELETING AN ITEM FROM THE LINKED LIST

To delete an existing node from a linked list, we have to loop through the nodes until we find the node we want to delete. We should follow the steps below to delete a node from a linked list:

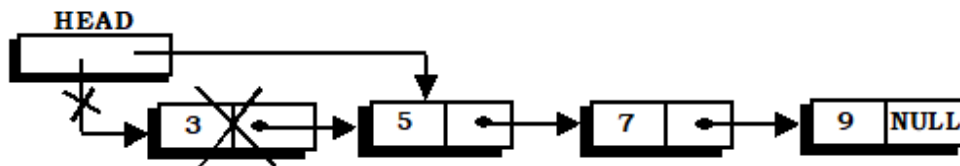
1. If the linked list is empty, then deletion is not possible and this condition is called as **underflow** condition.
2. To delete a particular node, we have to loop through the nodes until we find the node we want to delete.
3. If the end of the list has been reached, "**Position out of range**" message can be displayed.
4. Insert deleted node into the free area.

The deletion operation is classified into following types:

1. Deletion of the first node.
2. Deletion of the last node.
3. Deleting of the node at a given position.

DELETION OF FIRST NODE

To delete the first node, we just change the **HEAD** pointer to point to the second node or to contain NULL if we are deleting the only node in a one-node list. Suppose we have a linked list as shown in Figure and want to delete first node.



We don't show a complete code walk-through because the code is so straightforward. Instead, we show the state of the data structures in two stages: after the first two statements and at the end. We use one of our previous lists. Following is the data structure after the execution of the first two statements in the program segment. The following algorithm deletes the first node from the linked list and insert-deleted node into availability list.

ALGORITHM:

Let **LIST** be a linked list stored in memory. It deletes first node from the linked list. **HEAD** gives the address of the first node.

- Step 1 : [List empty ?]
 If HEAD = NULL
 Display "Under flow"
 Exit
 [EndIf]
- Step 2 : [Initialize pointer variable CurrPtr]
 CurrPtr = HEAD
- Step 3 : [Delete first node]
 HEAD = LINK[CurrPtr]
- Step 4 : [Return deleted node to the AVAIL list]
 FREE[CurrPtr]
- Step 5 : Exit

Function to delete a node at the beginning of the linked list

```
int Delete_Beginning()  
{  
    NODE *CurrPtr;  
    int ele;
```

```

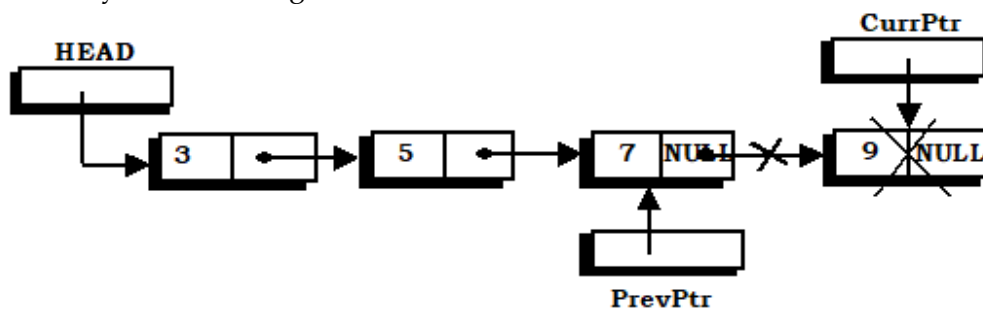
    if ( HEAD == NULL)
    {
        printf("\nDeletion not possible");
        exit(0);
    }
    CurrPtr = HEAD;
    ele = CurrPtr->info;
    HEAD = CurrPtr->link;

    free(CurrPtr);
    return(ele);
}

```

DELETION OF LAST NODE

To delete a last node of a linked list, we have to scan the list from the beginning and find the address of the next to last node and assign NULL to link field of the next to last node. The following algorithm deletes the last node from the linked list and insert-deleted node into availability list. Following is the data structure after the execution of the function.



ALGORITHM:

Let **LIST** be a linked list stored in memory, It deletes first node from the linked list. **HEAD** gives the address of the first node. **CurrPtr** is a pointer, which is being pointing current node of the list. **PrevPtr** is a pointer, which is pointing the previous node of the current node.

- Step 1 : [List empty ?]
 If HEAD = NULL Then
 Display "Under flow"
 Exit
 [EndIf]
- Step 2 : [Initialize pointer variables CurrPtr]
 CurrPtr = HEAD
- Step 3 : [Traverse linked list and take the pointer to next node]
 While (LINK(CurrPtr) != NULL) Do
- Step 4 : PrevPtr = CurrPtr
- Step 5 : CurrPtr = LINK[CurrPtr]
 [End of While]
- Step 6 : LINK[PrevPtr] = NULL
- Step 7 : [Return deleted node to the AVAIL list]

Step 8: FREE[CurrPtr]
 Exit

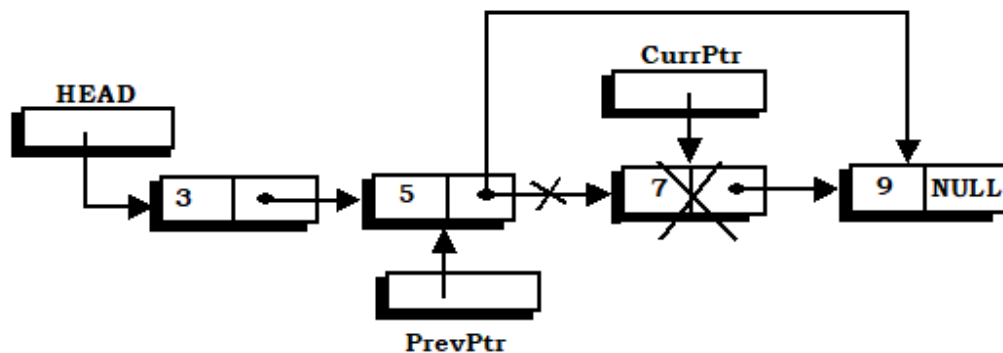
Function to delete the last node of the linked list

```
int Delete_End()  
{  
    NODE *CurrPtr,*PrevPtr;  
    int ele;  
  
    if ( HEAD == NULL)  
    {  
        printf("\nDeletion not possible");  
        exit(0);  
    }  
    CurrPtr = HEAD;  
    while ( CurrPtr->link != NULL)  
    {  
        PrevPtr = CurrPtr;  
        CurrPtr = CurrPtr->link;  
    }  
  
    ele = CurrPtr->info;  
    PrevPtr->link = NULL;  
  
    free(CurrPtr);  
    return(ele);  
}
```

DELETING THE NODE AT A GIVEN POSITION

The function for deleting a node at a given position involves the following steps.

1. Find the location of the node with **CurrPtr**, and location of previous node with **PrevPtr**.
2. Delete the node (This is nothing but deleting the node following a given node).



ALGORITHM:

Let **LIST** be a linked list stored in memory, It deletes first node from the linked list. **HEAD** gives the address of the first node. **POS** is the position of the node, which is to be deleted. **CurrPtr** is a pointer, which is being pointing to the node at a given position of the list. **PrevPtr** is a pointer, which is pointing the previous node of the current node.

```

Step 1 :      [List empty ?]
              If HEAD = NULL Then
                  Display "Under flow"
                  Exit
              [EndIf]
Step 2 :      [ Initialize pointer variables CurrPtr]
              CurrPtr = HEAD
Step 3 :      [Traverse linked list and take the pointer to the desired position]
              For I = 1 to POS and CurrPtr != NULL Do
Step 4 :      PrevPtr = CurrPtr
Step 5 :      CurrPtr = LINK[CurrPtr]
              [ End of For ]
Step 6 :      If CurrPtr = NULL Then
                  Display "Position out of range"
                  Exit
              [EndIf]
Step 7 :      [Delete the node]
              LINK(PrevPtr) = LINK(CurrPtr)
Step 8 :      [Return deleted node to the AVAIL list]
              FREE(CurrPtr)
Step 9 :      Exit

```

Function to delete a node at a given position of the linked list

```

int Delete_Position( int POS )
{
    NODE *CurrPtr,*PrevPtr;
    int ele,i;

    if ( HEAD == NULL)
    {
        printf("\nDeletion not possible");
        exit(0);
    }
    CurrPtr = HEAD;
    for( i = 1; i< POS && CurrPtr != NULL; i++)
    {
        PrevPtr = CurrPtr;
        CurrPtr = CurrPtr->link;
    }
}

```

```

        if ( CurrPtr == NULL)
        {
            printf("\nPositon out of range");
            exit(0);
        }
        ele = CurrPtr->info;
        PrevPtr->link = CurrPtr->link;

        free(CurrPtr);
        return(ele);
    }

```

Program: To create a linked list and delete an element from it

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <conio.h>
struct node
{
    int info;
    struct node *link;
};
typedef struct node NODE;
NODE *HEAD;
void Create_link_list(int);
void Display(NODE *);
int Delete_Beginning();
int Delete_End();
int Delete_Position(int);

void main( )
{
    int ele,pos;
    char ch;

    clrscr();
    HEAD = NULL;
    printf("\nLinked list creation\n\n");
    do{
        printf("Enter information field of the node\n");
        scanf("%d",&ele);
        Create_link_list(ele);
        printf("Add another node\n");
        fflush();
        scanf("%c",&ch);
    }while(toupper(ch)=='Y');
    printf("\nThe linked list is \n");

```

```

        Display(HEAD);
printf("\nDeletion Operation\n\n");
do
{
    printf("\nChoose Position To Delete: \n");
    printf("\n1. AT THE BEGINING");
    printf("\n2. AT THE END");
    printf("\n3. AT A GIVEN POSITION");
    printf("\n4. EXIT");
    printf("\n\nEnter your choice : ");
    scanf("%d",&ch);

    switch(ch)
    {
        case 1: {
                ele = Delete_Beginning();
                printf("\nDeleted element is :%d ",ele);
                printf("\nThe linked list is \n ");
                Display(HEAD);
                break;
            }

        case 2: {
                ele = Delete_End();
                printf("\nDeleted element is :%d ",ele);
                printf("\nThe linked list is \n ");
                Display(HEAD);
                break;
            }

        case 3: {
                printf("\nEnter position to delete: ");
                scanf("%d",&pos);
                ele = Delete_Position(pos);
                printf("\nDeleted element is :%d ",ele);
                printf("\nThe linked list is \n ");
                Display(HEAD);
                break;
            }

    }
}while(ch != 4);
getch();
}

```

/* Function to add a node to the linked list */

```

void Create_link_list(int ele)
{
    NODE *NewNode,*CurrPtr;

```

```

        NewNode=(NODE *)malloc(sizeof(NODE));
        NewNode->info=ele;
        NewNode->link=NULL;

        if( HEAD==NULL)
            HEAD=NewNode;
        else
        {
            CurrPtr=HEAD;
            while(CurrPtr->link!=NULL)
                CurrPtr=CurrPtr->link;

            CurrPtr->link=NewNode;
        }
    }
}

/* Function to display linked list elements */
void Display(NODE *CurrPtr)
{
    while(CurrPtr!=NULL)
    {
        printf("%d\t",CurrPtr->info);
        CurrPtr=CurrPtr->link;
    }
    printf("NULL\n");
}

/* Function to delete a node at the beginning of the linked list */
int Delete_Beginning( )
{
    NODE *CurrPtr;
    int ele;

    if ( HEAD == NULL)
    {
        printf("\nDeletion not possible");
        exit(0);
    }
    CurrPtr = HEAD;
    ele = CurrPtr->info;
    HEAD = CurrPtr->link;

    free(CurrPtr);
    return(ele);
}

```

/* Function to delete a node at the end of the linked list */

```
int Delete_End()  
{  
    NODE *CurrPtr,*PrevPtr;  
    int ele;  
  
    if ( HEAD == NULL)  
    {  
        printf("\nDeletion not possible");  
        exit(0);  
    }  
    CurrPtr = HEAD;  
    while ( CurrPtr->link != NULL)  
    {  
        PrevPtr = CurrPtr;  
        CurrPtr = CurrPtr->link;  
    }  
  
    ele = CurrPtr->info;  
    PrevPtr->link = NULL;  
  
    free(CurrPtr);  
    return(ele);  
}
```

/* Function to delete a node at a given position of the linked list */

```
int Delete_Position( int POS )  
{  
    NODE *CurrPtr,*PrevPtr;  
    int ele,i;  
  
    if ( HEAD == NULL)  
    {  
        printf("\nDeletion not possible");  
        exit(0);  
    }  
    CurrPtr = HEAD;  
    for( i = 1; i< POS && CurrPtr != NULL; i++)  
    {  
        PrevPtr = CurrPtr;  
        CurrPtr = CurrPtr->link;  
    }  
    if ( CurrPtr == NULL)  
    {  
        printf("\nPositon out of range");  
        exit(0);  
    }  
}
```

```

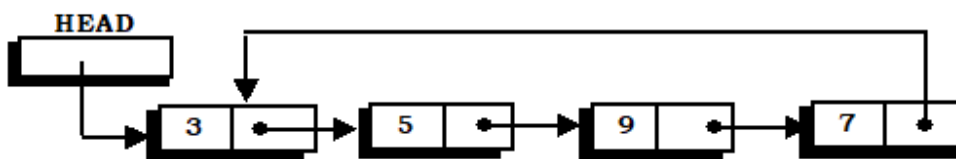
    }
    ele = CurrPtr->info;
    PrevPtr->link = CurrPtr->link;

    free(CurrPtr);
    return(ele);
}

```

CIRCULAR LINKED LINEAR LIST

Some applications do not require that there be a particular first or last list element. In such cases, it may be convenient to allow access from the last element to the first. In the normal linked list, the link field of the last element stores the value **NULL**. By instead storing a pointer to the first element in the list, a **circular list** is created. With this implementation, a **TAIL** pointer is no longer needed. Figure illustrates a circular singly linked list. The primary danger with such implementations is that list-processing operations may go into infinite loop since there is no obvious end to the list. However, the **HEAD** pointer can be used as a marker to determine when list processing has worked full circle through the list.



ADVANTAGES OF CIRCULAR LINKED LIST

1. In a circular linked list every node is accessible from a given node. That is from the given node, nodes can be reached by merely chaining the list.
2. Concatenation and splitting the operations are more efficient.

DISADVANTAGES OF CIRCULAR LINKED LIST

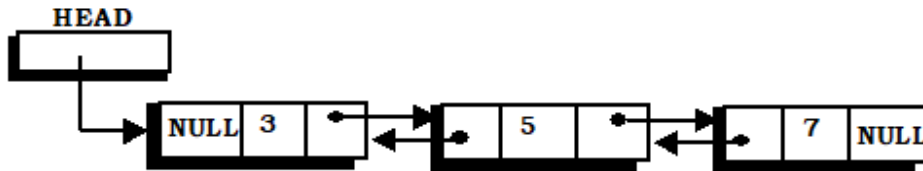
1. In processing, it is possible to get into an infinite loop.

DOUBLY LINKED LIST

The singly linked list presented in previous section allows for direct access from a list node only to the next node in the list. This list is named as singly linked list because each list element contains a pointer to the next element. In singly linked list, traversing is possible only in one direction. Sometimes it is required to traverse a list in forward direction or backward direction. A doubly linked list is designed to allow convenient access from a list node to the next node and also to the preceding node on the list. This is also used to traverse the list in both the directions.

In a doubly linked list each node has not only pointer to the next node but also a pointer to the prior node. Each node of doubly linked list is divided into three parts (fields):

1. A pointer BACK (leftlink) which contains the location of the preceding node in the list.
2. An information field INFO which contains data of N.
3. A pointer FORW (rightlink) which contains location of the next node in the list.



The doubly linked list is also called as **two-way** list because we can traverse the list in two directions: in the usual forward direction from the beginning of the list to end, or in the backward direction from the end of the list to the beginning.

The doubly linked list is useful primarily because it is easier to implement operations than a singly linked list. We may consider the nodes on a doubly linked list to consists of three fields: an info field that contains the information stored in the node, and left and right fields that contain pointers to the nodes on either side. In the figure, **LEFT** and **RIGHT** are pointer variables denoting the left-most and right-most nodes in the list, respectively. The left link of the left-most node and the right link of the right-most node are both **NULL**, indicating the end of the list for each direction. The left and right links of a node are denoted by the variables **llink** and **rlink**, respectively. We may declare a set of such nodes using dynamic implementation, by

```

struct node
{
    int info;           // Information field
    struct node *rlink; // a pointer which is pointing right node
    struct node *llink; // a pointer which is pointing left node
};
  
```

ADVANTAGES OF DOUBLY LINKED LIST

1. One can traverse the list in both directions.
2. Insertions and deletions are easy.
3. It is used to represent trees.

DISADVANTAGE OF DOUBLY LINKED LIST

1. Extra memory is required to store RIGHT and LEFT pointers.

MERITS OF AN ARRAY OVER LINKED LIST

1. It is easy to compute the address of an element in an array.
2. Linear relationship is reflected by the physical relationship of the data in memory.
3. Extra memory space is not required to store link fields.
4. Binary search is applicable
5. Sorting an array is easier.
6. Once can directly access any element of the array.

DEMERITS OVER LINKED LIST

1. Additions and deletions are not easy.
2. It cannot be used in application where there is an unpredictable storage requirement.

APPLICATIONS OF THE LINKED LIST

This section will be concerned with a number of applications of linear linked lists. Many examples could be given, but only first two applications will be described in this section.

1. It is used to represent a polynomial.
2. It is used to find the sum of two long integers.
3. It is used to represent queues, stacks in memory resulting in efficient use of storage and computer time.
4. It is used in symbol table construction.
5. It is used in memory management of operating systems.